

# Code With Me: Complex Data Types in C

## MTU – C Programming Night Class

**Topics:** structs · pointers · nested structs · linked lists · enums · unions · typedef

### What each demo covers:

Demo 1 — struct definition, dot operator, constant initialisers, designated initialisers, and struct assignment

Demo 2 — pointer to struct, why `*ptr.name` is wrong, and the arrow operator `->` vs `(*ptr).name`

Demo 3 — pass-by-value with structs (the function modifies a copy, the original is unchanged)

Demo 4 — enum constants with explicit values and a switch statement

Demo 5 — tagged union with the task of uncommenting the broken line so students see the memory overwrite happen

Demo 6 — nested structs with chained dot and arrow operators

Demo 7 — the full linked list: building, inserting, removing, searching, and freeing

---

### ***How to use this worksheet***

*Work through each section one at a time. Type out the code yourself — don't copy and paste. Compile and run after each section before moving on.*

```
gcc -Wall -o complex_types complex_types.c
./complex_types
```

---

## Section 1 – Defining a **struct**

A **struct** groups related fields under one type name. Think of it like a Java class — but with no methods.

```
struct student {
    char *name;
    int    id;
    int    year;
};
```

### Declaring variables:

```
struct student mike;           /* single variable */
struct student cohort[30];     /* array of students */
```

### Initialising field by field:

```
mike.name = "Mike";
mike.id   = 101;
mike.year = 1;
```

### Struct constant initialiser:

```
struct student sarah = {"Sarah", 102, 2};
```

### Designated initialiser (only sets chosen fields):

```
struct student anon = { .name = "TBD" };
```

### Struct assignment (unlike arrays, this is allowed):

```
struct student copy = sarah; /* all fields are copied */
```

## Tasks

1. Add a fourth field `char *course` to the struct and initialise it for both `mike` and `sarah`.
2. Print all four fields for both students using `printf`.
3. Assign `sarah` to a new variable `john` and change `john.name` to `"John"`. Does `sarah.name` change? Why or why not?

---

## Section 2 – `typedef` — Cleaner Type Names

Instead of writing `struct student` every time, `typedef` creates an alias.

```
typedef struct student Student;

/* Now both of these are equivalent: */
struct student mike;
Student      mike;
```

You can merge the **struct** definition and **typedef** in one step:

```
typedef struct {
    char *name;
    int   id;
    int   year;
} Student;
```

## Tasks

1. Create a **typedef** for a struct called **Point** with two **int** fields: **x** and **y**.
2. Declare a **Point** variable, set **x = 3** and **y = 7**, and print both values.

---

## Section 3 – Pointer to Struct & the Arrow Operator

When you have a **pointer** to a struct, the dot operator **.** does not work directly.

```
Student dave = {"Dave", 103, 3};
Student *ptr  = &dave;
```

| Attempt             | Result                                                   |
|---------------------|----------------------------------------------------------|
| <b>*ptr.name</b>    | ✗ Wrong — parsed as <b>*(ptr.name)</b> due to precedence |
| <b>(*ptr).name</b>  | ✓ Correct — dereference first, then access field         |
| <b>ptr-&gt;name</b> | ✓ Correct — shorthand for the above, preferred style     |

```
/* Both lines below are equivalent */
printf("%s\n", (*ptr).name);
printf("%s\n", ptr->name);
```

**Modifying through a pointer affects the original:**

```
ptr->year = 4;
printf("%d\n", dave.year);    /* prints 4 – original is changed */
```

## Tasks

1. Declare a **Student** and a pointer to it. Use **->** to change the **id** field. Print the original variable to confirm it changed.
2. Try writing **\*ptr.name** and compile. Read the compiler warning or error. What does it say?

---

## Section 4 – Structs Are Passed By Value

When you pass a struct to a function, a **copy** is made. Changes inside the function do not affect the original.

```
void try_increment(Student s) {
    s.year++;
    printf("Inside function: year = %d\n", s.year);
}

int main(void) {
    Student tom = {"Tom", 104, 1};
    printf("Before: year = %d\n", tom.year);
    try_increment(tom);
    printf("After : year = %d\n", tom.year);    /* still 1 */
    return 0;
}
```

**Output:**

```
Before: year = 1
Inside function: year = 2
After : year = 1
```

To modify the original, pass a **pointer to the struct**:

```
void increment_year(Student *s) {
    s->year++;
}

increment_year(&tom);
printf("%d\n", tom.year);    /* now 2 */
```

## Tasks

1. Write a function `void reset_id(Student s)` that sets `s.id = 0`. Call it and check — does the original change?
2. Write a second version `void reset_id_ptr(Student *s)` that does the same through a pointer. Does the original change now?
3. Explain in one sentence why the two versions behave differently.

---

## Section 5 – Enums

An `enum` gives meaningful names to a set of integer constants.

```
typedef enum {
    YEAR_ONE = 1,
    YEAR_TWO,      /* automatically 2 */
    YEAR_THREE,    /* automatically 3 */
    YEAR_FOUR      /* automatically 4 */
} AcademicYear;
```

Using an enum in a `switch` :

```
AcademicYear yr = YEAR_TWO;

switch (yr) {
    case YEAR_ONE:    printf("Year 1\n"); break;
    case YEAR_TWO:    printf("Year 2\n"); break;
    case YEAR_THREE:  printf("Year 3\n"); break;
    case YEAR_FOUR:   printf("Year 4\n"); break;
}
```

```
/* Enum constants are just integers underneath */
printf("YEAR_ONE    = %d\n", YEAR_ONE);    /* 1 */
printf("YEAR_THREE  = %d\n", YEAR_THREE);  /* 3 */
```

## Tasks

1. Add `YEAR_FIVE` to the enum. What value does it get automatically?
2. Create an enum called `Grade` with values `PASS`, `MERIT`, `DISTINCTION`. What integer values do these get by default (starting from 0)?
3. Change `Grade` so `PASS = 40`, `MERIT = 60`, `DISTINCTION = 70`. Print all three values.

---

## Section 6 – Unions

A union looks like a struct but all members **share the same memory**. Only **one field is valid at a time**.

```
union Address {
    int  tcp_port;
    char ipv4[4];
};

union Address addr;

addr.tcp_port = 7;
printf("%d\n", addr.tcp_port);    /* 7 */
```

```

addr.ipv4[0] = 192; addr.ipv4[1] = 168;
addr.ipv4[2] = 1;   addr.ipv4[3] = 10;
printf("%d.%d.%d.%d\n",
    addr.ipv4[0], addr.ipv4[1],
    addr.ipv4[2], addr.ipv4[3]); /* 192.168.1.10 */

printf("%d\n", addr.tcp_port); /* ⚠ NOT 7 anymore – memory was overwritten */

```

### Tagged union — track which field is active:

```

typedef enum { CONTACT_PHONE, CONTACT_EMAIL } ContactType;

typedef struct {
    ContactType type;
    union {
        long phone;
        char email[50];
    } detail;
} ContactDetail;

ContactDetail c;
c.type = CONTACT_EMAIL;
strcpy(c.detail.email, "mike@mtu.ie");

if (c.type == CONTACT_EMAIL)
    printf("Email: %s\n", c.detail.email);

```

## Tasks

1. Set `c.detail.phone = 0851234567`, then immediately set `c.detail.email = "test@mtu.ie"`. Print `c.detail.phone` after. What value do you get and why?
2. In your own words: what is the difference in memory layout between a `union` and a `struct`?

3. Why is the `type` field (the tag) important when using a union?

---

## Section 7 – Nested Structs

Structs can contain other structs — either embedded directly (1:1) or via a pointer.

```
typedef struct {
    char street[50];
    char city[30];
} Address;

typedef struct {
    char      *name;
    int       id;
    AcademicYear year;
    Address    address;      /* embedded – 1:1 */
    ContactDetail contact;   /* tagged union contact */
} FullStudent;
```

**Accessing nested fields — chain the operators:**

```
FullStudent fs;
fs.name = "Emma";
fs.id   = 201;
fs.year = YEAR_THREE;

strcpy(fs.address.street, "123 Main Street");
strcpy(fs.address.city,   "Cork");

fs.contact.type = CONTACT_EMAIL;
strcpy(fs.contact.detail.email, "emma@student.mtu.ie");

printf("Name    : %s\n", fs.name);
```

```
printf("City   : %s\n", fs.address.city);
printf("Email  : %s\n", fs.contact.detail.email);
```

When the nested struct is a pointer, use **->** at that level:

```
typedef struct {
    char    *name;
    Address *address;      /* pointer to Address */
} SlimStudent;

SlimStudent ss;
Address      home = {"45 Bridge St", "Limerick"};
ss.name      = "Liam";
ss.address   = &home;

printf("%s\n", ss.address->city);  /* dot to reach pointer, arrow to dereference */
```

## Tasks

1. Add a **char \*country** field to **Address** . Set it for **fs** and print it.
2. Create a **SlimStudent** , allocate its **Address** with **malloc** , populate it, and print the city using **->** .
3. Draw a memory diagram showing the difference between an embedded **Address** and an **Address \*** pointer inside a struct.

---

## Section 8 – Linked Lists

A linked list is built from **self-referential nodes** — each node points to the next.

```
typedef struct Node {
    void    *content;  /* pointer to any data type */
    struct Node *next;  /* pointer to next node    */
} Node;
```

## 8.1 – Creating the First Node

```
Node *list_init(void *content) {  
    Node *head    = malloc(sizeof(Node));  
    head->content = content;  
    head->next     = NULL;    /* no next node yet */  
    return head;  
}
```

## 8.2 – Adding a Node at the End

```
Node *list_add(Node *head, void *content) {  
    Node *new_node    = malloc(sizeof(Node));  
    new_node->content = content;  
    new_node->next     = NULL;  
  
    Node *index = head;  
    while (index->next != NULL) {    /* walk to last node */  
        index = index->next;  
    }  
    index->next = new_node;  
    return new_node;  
}
```

## 8.3 – Inserting a Node in the Middle

```
/* Insert new node AFTER the node pointed to by 'index' */  
Node *list_insert_after(Node *index, void *content) {  
    Node *new_node    = malloc(sizeof(Node));  
    new_node->content = content;  
    new_node->next     = index->next;    /* Step 1: forward link */  
    index->next        = new_node;      /* Step 2: backward link */  
}
```

```
    return new_node;
}
```

⚠ **Order matters!** Step 1 *MUST* happen before Step 2.

*If you reverse them you lose the reference to the rest of the list permanently.*

## 8.4 – Searching the List

```
Node *index = head;
while (index != NULL) {
    Student *s = (Student *)index->content;
    if (s->id == 305) {
        printf("Found: %s\n", s->name);
        break;
    }
    index = index->next;
}
if (index == NULL) printf("Not found.\n");
```

## 8.5 – Removing a Node

```
Node *list_remove(Node *head, Node *target) {
    if (head == target) {
        Node *new_head = head->next;
        free(target);
        return new_head;
    }
    Node *index = head;
    while (index->next != NULL && index->next != target) {
        index = index->next;
    }
    if (index->next == target) {
        index->next = target->next;    /* bypass target */
        free(target);                /* release memory */
    }
}
```

```
}  
  
return head;  
  
}
```

## 8.6 – Putting It All Together

```
int main(void) {  
    /* Create students */  
    Student *s1 = malloc(sizeof(Student));  
    s1->name = "Alice"; s1->id = 301; s1->year = YEAR_ONE;  
  
    Student *s2 = malloc(sizeof(Student));  
    s2->name = "Bob";    s2->id = 302; s2->year = YEAR_TWO;  
  
    Student *s3 = malloc(sizeof(Student));  
    s3->name = "Charlie"; s3->id = 303; s3->year = YEAR_THREE;  
  
    /* Build list */  
    Node *head = list_init(s1);  
    Node *n2    = list_add(head, s2);  
                list_add(head, s3);  
  
    /* Insert Diana after Bob */  
    Student *s4 = malloc(sizeof(Student));  
    s4->name = "Diana"; s4->id = 304; s4->year = YEAR_TWO;  
    list_insert_after(n2, s4);  
  
    /* Print list */  
    Node *idx = head;  
    while (idx != NULL) {  
        Student *s = (Student *)idx->content;  
        printf("ID: %d | Name: %s\n", s->id, s->name);  
        idx = idx->next;  
    }  
}
```

```
return 0;
}
```

Expected output:

```
ID: 301 | Name: Alice
ID: 302 | Name: Bob
ID: 304 | Name: Diana
ID: 303 | Name: Charlie
```

 Tasks

- 1. Draw the memory diagram for the list after all four students are added (before any removal).
- 2. Add a fifth student **Eve** at the end and print the list again.
- 3. Remove **Bob** from the list using `list_remove` . Print the list to confirm.
- 4. Write a function `int list_count(Node *head)` that returns the number of nodes in the list.
- 5. **Challenge:** Write a function `Node *list_find_by_id(Node *head, int id)` that returns the node containing the student with the matching ID, or `NULL` if not found.

Summary

| Concept              | Key Rule                                                                                      |
|----------------------|-----------------------------------------------------------------------------------------------|
| <code>struct</code>  | Groups related fields. Use <code>.</code> for variables, <code>-&gt;</code> for pointers.     |
| <code>typedef</code> | Creates a cleaner alias for a type ( <code>struct student</code> → <code>Student</code> ).    |
| Structs in functions | Passed <b>by value</b> — use a pointer parameter to modify the original.                      |
| <code>enum</code>    | Named integer constants. Values auto-increment unless set explicitly.                         |
| <code>union</code>   | All members share the <b>same memory</b> . Only one is valid at a time.                       |
| Tagged union         | Pair a <code>union</code> with an <code>enum</code> to track which field is currently active. |
| Nested structs       | Use <code>.</code> for embedded structs, <code>-&gt;</code> when accessing through a pointer. |

| Concept      | Key Rule                                                                                           |
|--------------|----------------------------------------------------------------------------------------------------|
| Linked list  | Nodes linked by <code>next</code> pointers. Always set <code>next = NULL</code> for the last node. |
| Insert order | When inserting a node: connect <b>forward first</b> , then <b>backward</b> .                       |
| Remove       | Rewire the previous node's <code>next</code> , then <code>free()</code> the removed node.          |